

Taller 2: Fundamentos "Pythonic" Estructuras Core

Docente: Dudbil Olvasada Pabon Riaño

1. Objetivo del Taller

Desarrollar la lógica de programación mediante el dominio de las estructuras de datos nativas de Python (Listas, Tuplas, Diccionarios y Sets). El estudiante aprenderá a escribir código eficiente, legible y alineado con la filosofía "Pythonic" de Joel Grus, sentando las bases para la posterior manipulación de DataFrames masivos.

2. Instrucciones Generales

Desarrolle los siguientes retos en un Jupyter Notebook (Google Colab). Utilice celdas de Markdown para explicar la lógica de su código. **Restricción estricta:** En este taller está prohibido importar librerías externas como Pandas o Numpy. Toda solución debe construirse usando exclusivamente Python puro (Built-in functions). Al finalizar, descargue el notebook o suba el enlace a su repositorio.

Misión Analítica: El Motor del E-Commerce

Ha sido asignado al equipo de ingeniería de datos de una startup de comercio electrónico. Su tarea es procesar las estructuras de datos crudas (JSON-like) provenientes de la base de datos de usuarios y carritos de compras, para generar analítica de valor.

Reto 1: La Magia de las List Comprehensions (20 %)

El departamento de ventas necesita aplicar un descuento relámpago del 15 % a todos los productos de la tienda y filtrar solo aquellos que queden con un precio final menor a \$50. Dado el siguiente listado de precios (en dólares):

```
precios = [25.5, 60.0, 15.0, 120.0, 45.0, 99.9, 10.0]
```

1. Resuelva el problema escribiendo un **bucle for tradicional** (creando una lista vacía y haciendo **append**).
2. Resuelva el **mismo problema** en una sola línea de código utilizando una **List Comprehension**.
3. **Análisis Crítico:** Basado en el "Zen de Python", ¿por qué la segunda opción es preferida por los Científicos de Datos?

Reto 2: El Diccionario como Base de Datos (30 %)

Usted recibe la información de un usuario desde la API en formato Diccionario:

```
usuario = {  
    "id": 105,  
    "nombre": "Ana Gomez",  
    "historial_compras": [  
        {"item": "Laptop", "precio": 1200},  
        {"item": "Mouse", "precio": 25},  
        {"item": "Monitor", "precio": 250}  
    ],  
    "activo": True  
}
```

1. Escriba el código en Python para extraer (imprimir) el precio del "Monitor" navegando por la estructura anidada del diccionario.
2. Calcule el "Total Gastado" por la usuaria iterando sobre su `historial_compras` (recuerde: es una lista de diccionarios).
3. Agregue una nueva clave al diccionario principal llamada `categoria_cliente` que sea "VIP" si el gasto total supera los \$1000, y "Standard" si es menor. Imprima el diccionario actualizado.

Reto 3: Desduplicación con Sets (20 %)

El equipo de marketing lanzó una campaña y registró los IDs de los usuarios que hicieron clic en el anuncio. Sin embargo, el sistema falló y guardó clics duplicados del mismo usuario.

```
clics = [101, 102, 101, 105, 108, 102, 109, 101, 110]
```

1. Utilizando la estructura de datos **Set** (Conjunto), limpie la lista para que contenga únicamente IDs únicos.
2. Calcule y muestre en pantalla cuántos usuarios **únicos** interactuaron con la campaña.

Reto 4: Tuplas y Desempaquetado (Tuple Unpacking) (10 %)

Usted tiene una función (simulada) de una base de datos que retorna las coordenadas espaciales de los repartidores como una **Tupla**: `coordenadas = (4.6097, -74.0817, "Bogotá")`

1. Utilice la técnica de "Tuple Unpacking" para asignar estos tres valores directamente a tres variables distintas (`latitud`, `longitud`, `ciudad`) en una sola línea de código.
2. Intente reasignar el valor de la latitud modificando la tupla original (Ej. `coordenadas[0] = 5.0`). Explique el error que Python arroja y la importancia de la **inmutabilidad** en el diseño de software.

Reto 5: Reto Autónomo de Integración (20 %)

Cree un programa que reciba un texto corto (Ej. "la ciencia de datos es el futuro de la ciencia"). Su objetivo es construir un diccionario donde las **claves** sean las palabras únicas del texto, y los **valores** sean la cantidad de veces que aparece esa palabra (frecuencia). *Pista: Puede usar el método `.split()` para separar el texto en una lista de palabras, y luego iterar sobre ella.*

"Hermoso es mejor que feo. Explícito es mejor que implícito. Simple es mejor que complejo."

- El Zen de Python